CSS Tutorial

A Complete Guide

14 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

https://snehaliteng.github.io/

Copyright & Disclaimer

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead: https://snehaliteng.github.io/tutorials/css/

CHAPTER 1

Getting Started

What is CSS?

CSS (Cascading Style Sheets) is the language used to style HTML documents. It controls the layout, colors, fonts, and overall visual appearance of a web page.

CSS Syntax

A CSS rule consists of a **selector** and a **declaration block**:

```
selector {
  property: value;
}
```

Example

```
h1 {
  color: blue;
  font-size: 24px;
}
```

> **Tip:** Always end each declaration with a semicolon. Missing semicolons are a common cause of broken styles.

Types of CSS

Inline CSS

Applied directly to an element using the `style` attribute.

```
<p style="color: red;">This is red text.</p>
```

Internal CSS

Written inside a `<style>` tag in the `<head>` section.

```
<style>
  p { color: red; }
</style>
```

External CSS

Linked via a `.css` file using the `<link>` tag. This is the recommended approach.

```
<link rel="stylesheet" href="styles.css">
```

CSS Selectors

Element Selector

```
p {
  color: green;
}
```

Class Selector

```
.highlight {
  background: yellow;
}
```

ID Selector

```
#header {
  font-size: 2rem;
}
```

Universal Selector

```
* {
  margin: 0;
  padding: 0;
}
```

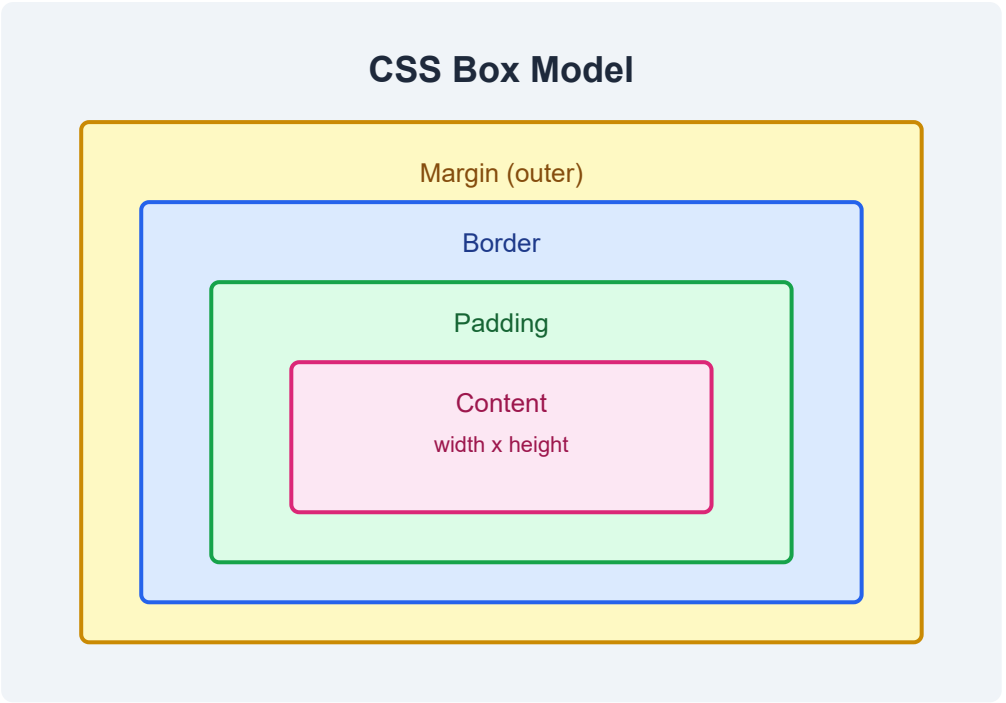
CSS Comments

Comments are ignored by the browser and help document your code:

```
/* This is a CSS comment */
h1 {
  color: navy;
}
```

Common Errors

- Missing closing curly braces or semicolons.
- Misspelled property names.
- Using incorrect selector syntax.
- Forgetting to link the external stylesheet.



Practice Task

Create an HTML page with an external stylesheet. Style a `<h1>` with a color of your choice and a `<p>` with a class of `intro` that has a larger font size.

CHAPTER 2

Colors & Backgrounds

CSS Colors

Colors can be specified in several ways:

Named Colors

```
h1 { color: red; }
p { color: navy; }
```

Hex Colors

```
h1 { color: #ff0000; }
p { color: #000080; }
```

RGB and RGBA

```
h1 { color: rgb(255, 0, 0); }
div { background: rgba(0, 0, 255, 0.5); }
```

HSL

```
h1 { color: hsl(0, 100%, 50%); }
div { background: hsla(240, 100%, 50%, 0.5); }
```

Background Properties

background-color

```
body { background-color: #f4f4f4; }
```

background-image

```
header {
  background-image: url("banner.jpg");
}
```

Borders

Borders have three properties: width, style, and color.

```
div {
  border-width: 2px;
  border-style: solid;
  border-color: #333;
}
```

Shorthand

```
div {
  border: 2px solid #333;
}
```

Margins and Padding

Margin creates space outside the element. **Padding** creates space inside the element, between the content and the border.

```
div {
  margin: 20px;
  padding: 15px;
}
```

Height and Width

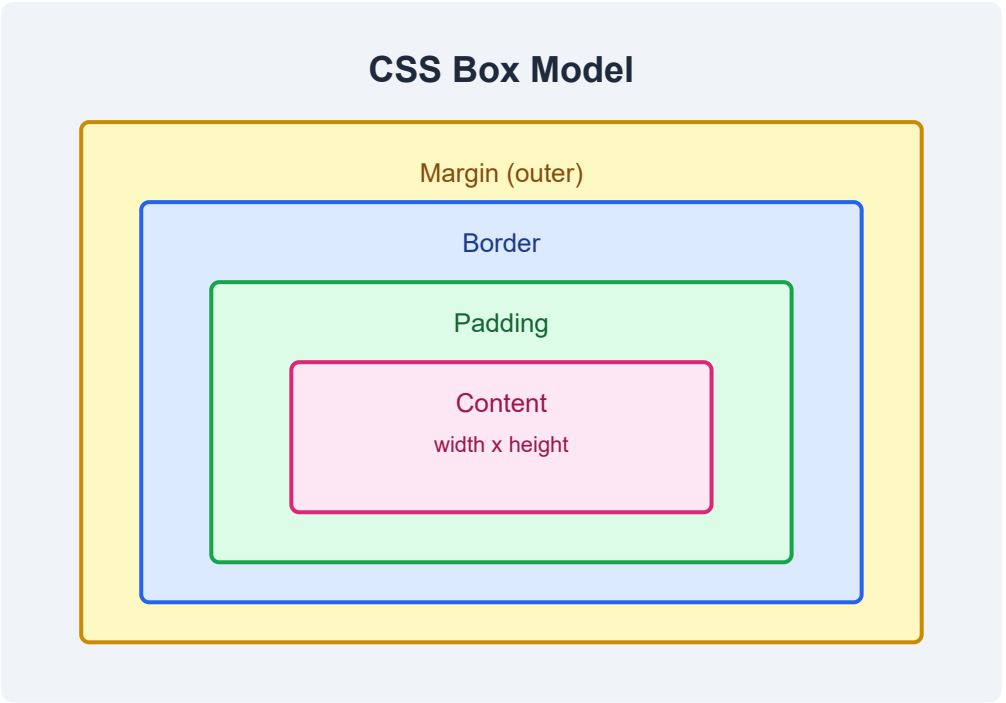
```
div {
  height: 200px;
  width: 50%;
}
```

Outline

An outline is drawn outside the border edge and does not affect layout.

```
input:focus {
  outline: 2px solid blue;
}
```

> **Note:** Unlike borders, outlines do not take up space and may overlap other elements.



Practice Task

Create a card component with a background color, a solid border, rounded corners (use `border-radius`), padding inside, and a margin around it.

CHAPTER 3

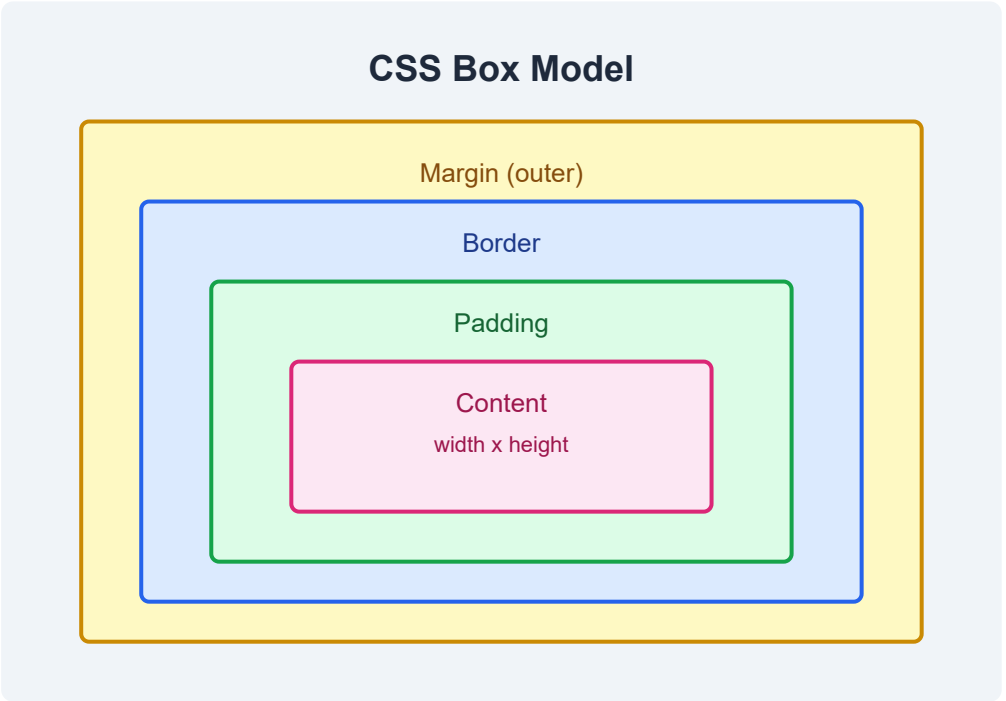
Box Model

What is the Box Model?

Every element in CSS is a rectangular box. The box model describes how the size of each element is calculated from four layers: content, padding, border, and margin.

The Four Areas

- **Content** — the actual text or image.
- **Padding** — space between content and border.
- **Border** — the edge around the padding.
- **Margin** — space outside the border.



box-sizing

The `box-sizing` property controls how width and height are calculated.

content-box (default)

Width and height only apply to the content area. Padding and border add to the total size.

```
div {
  box-sizing: content-box;
  width: 200px;      /* content width */
  padding: 20px;     /* adds 40px total */
  border: 2px solid; /* adds 4px total */
  /* total width: 244px */
}
```

border-box

Width and height include content, padding, and border. This makes sizing much easier.

```
div {
  box-sizing: border-box;
  width: 200px;      /* total width */
  padding: 20px;
  border: 2px solid;
  /* content width: 156px */
}
```

Best practice: Set `box-sizing: border-box` on all elements with the universal selector to simplify layout math:

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

Margin Collapse

When two vertical margins meet, the larger margin wins. They do not add together.

```
h1 { margin-bottom: 30px; }
h2 { margin-top: 20px; }
/* The gap between them is 30px, not 50px */
```

Block vs Inline Boxes

- **Block** elements take the full width available and start on a new line.
- **Inline** elements take only as much width as needed and flow within text.

Practice Task

Create two `<div>` elements side by side. Set each to `width: 50%` with `box-sizing: border-box` and add padding and borders. Observe how `border-box` keeps them on the same line.

CHAPTER 4

Layout & Positioning

The display Property

The `display` property controls how an element behaves in the layout.

block

```
p { display: block; }
```

inline

```
span { display: inline; }
```

inline-block

```
button { display: inline-block; }
```

none

```
.hidden { display: none; }
```

> **Note:** `visibility: hidden` hides the element but keeps its space. `display: none` removes the element entirely from the layout.

max-width

```
div {
  max-width: 800px;
  margin: 0 auto;
}
```

Positioning

static (default)

Elements appear in normal document flow. The `top`, `right`, `bottom`, and `left` properties have no effect.

relative

Offset from its normal position without affecting surrounding elements.

```
div {
  position: relative;
  top: 10px;
  left: 20px;
}
```

absolute

Positioned relative to the nearest positioned ancestor. Removed from normal flow.

```
div {
  position: absolute;
  top: 0;
  right: 0;
}
```

fixed

Positioned relative to the viewport. Stays in place during scrolling.

```
nav {
  position: fixed;
  top: 0;
  width: 100%;
}
```

sticky

Toggles between relative and fixed depending on scroll position.

```
header {
  position: sticky;
  top: 0;
}
```

z-index

Controls the stacking order of positioned elements. Higher values appear on top.

```
.overlay {
  position: absolute;
  z-index: 10;
}
```

Overflow

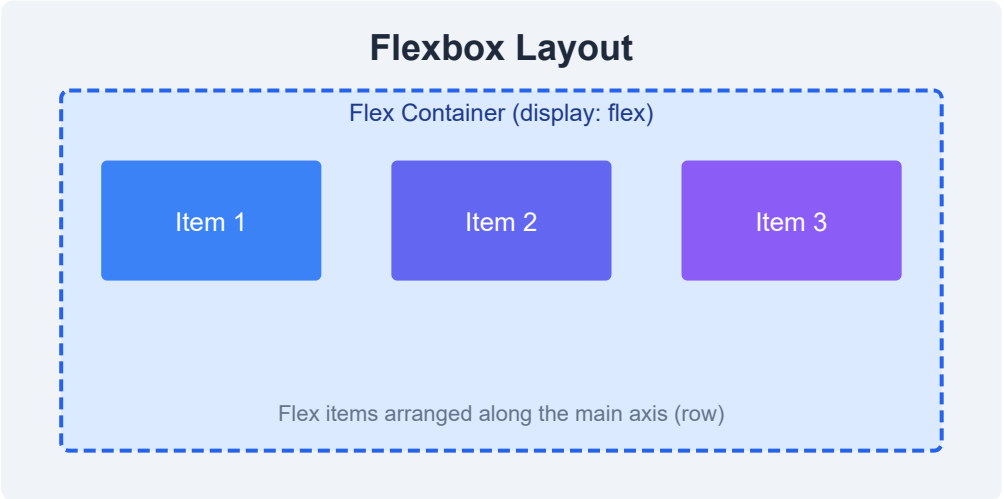
```
div {
  overflow: hidden;   /* clip content */
  overflow: scroll;   /* always show scrollbars */
  overflow: auto;     /* show scrollbars only when needed */
}
```

Float and Clear

```
img {
  float: left;
  margin-right: 10px;
}

footer {
  clear: both;
}
```

Alignment

```
p { text-align: center; }

div {
  width: 50%;
  margin: 0 auto;    /* center block elements */
}
```



Practice Task

Create a fixed header that stays at the top of the page, a sidebar with `position: sticky`, and a main content area that scrolls beneath them.

CHAPTER 5

Selectors & Effects

Combinators

Combinators define relationships between selectors.

Descendant Selector (space)

Selects all matching descendants regardless of depth.

```
article p {
  color: gray;
}
```

Child Selector (>)

Selects only direct children.

```
ul > li {
  list-style: none;
}
```

Adjacent Sibling Selector (+)

Selects an element immediately preceded by a specific sibling.

```
h2 + p {
  font-weight: bold;
}
```

General Sibling Selector (~)

Selects all siblings that follow a specific element.

```
h2 ~ p {
  color: #666;
}
```

Pseudo-classes

Pseudo-classes describe a special state of an element.

:hover

```
a:hover {
  color: orange;
}
```

:focus

```
input:focus {
  border-color: blue;
}
```

:nth-child

```
li:nth-child(odd) {
  background: #f0f0f0;
}

li:nth-child(3n) {
  border-bottom: 1px solid #ccc;
}
```

:first-child and :last-child

```
p:first-child {
  margin-top: 0;
}

p:last-child {
  margin-bottom: 0;
}
```

Pseudo-elements

Pseudo-elements target a specific part of an element.

::before and ::after

```
.note::before {
  content: "\26A0";
  margin-right: 5px;
}

.link::after {
  content: " \2197";
}
```

::first-line

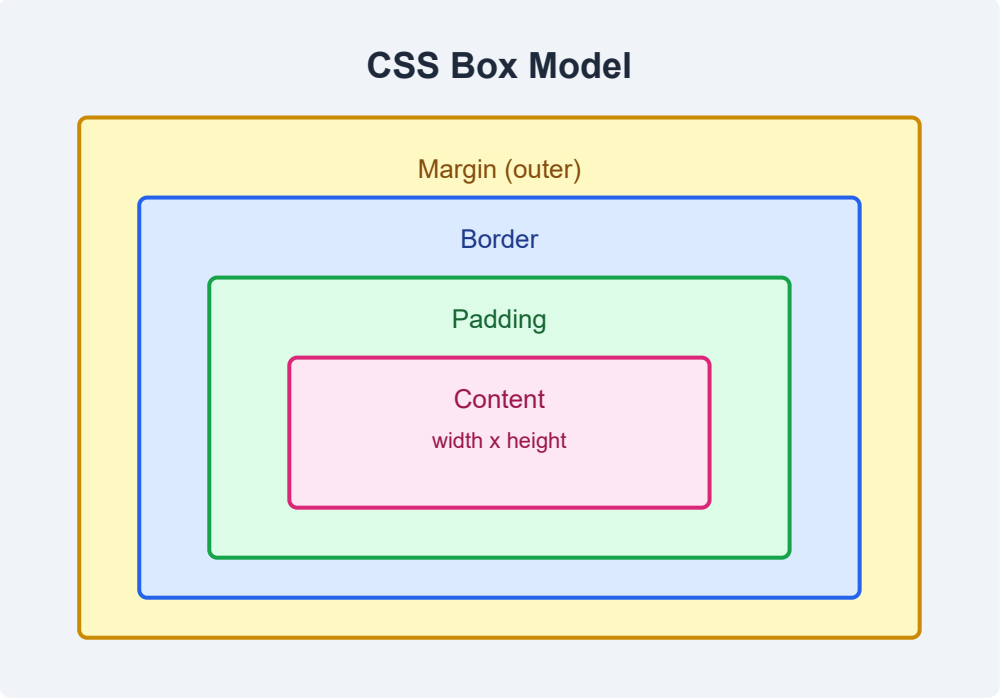
```
p::first-line {
  font-variant: small-caps;
}
```

Attribute Selectors

```
/* Exact match */
input[type="text"] { border: 1px solid gray; }

/* Contains */
a[href*="example"] { color: green; }

/* Starts with */
a[href^="https"] { font-weight: bold; }

/* Ends with */
img[src$=".png"] { border-radius: 4px; }
```

Opacity

```
.faded {
  opacity: 0.5;
}
```



Practice Task

Style a table with alternating row colors using `:nth-child(even)`, add an icon before each link using `::before`, and apply a hover effect that changes background color.

CHAPTER 6

Navigation & UI

Navigation Bars

Vertical Navigation

```
nav ul {
  list-style: none;
  padding: 0;
}

nav a {
  display: block;
  padding: 10px;
  text-decoration: none;
}
```

Horizontal Navigation

```
nav li {
  display: inline-block;
}

nav a {
  padding: 10px 20px;
}
```

Dropdown Menus

```
.dropdown {
  position: relative;
  display: inline-block;
}

.dropdown-content {
  display: none;
  position: absolute;
}

.dropdown:hover .dropdown-content {
  display: block;
}
```

Image Galleries

```
.gallery {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}

.gallery img {
  width: 200px;
  height: 150px;
  object-fit: cover;
}
```

Image Sprites

A single image containing multiple icons is used as a sprite. The `background-position` property shifts the visible portion.

```css
.icon-home {
  width: 32px;
  height: 32px;
  background: url("sprites.png") 0 0;
}

.icon-settings {
  background: url("sprites.png") -32px 0;
}
```

Form Styling

```css
input[type="text"],
textarea {
  width: 100%;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
}

button {
  background: #3b82f6;
  color: #fff;
  padding: 10px 20px;
  border: none;
  cursor: pointer;
}

button:hover {
  background: #2563eb;
}
```

CSS Counters

```
ol {
  counter-reset: section;
}

li::before {
  counter-increment: section;
  content: "Section " counter(section) ": ";
}
```

CSS Units

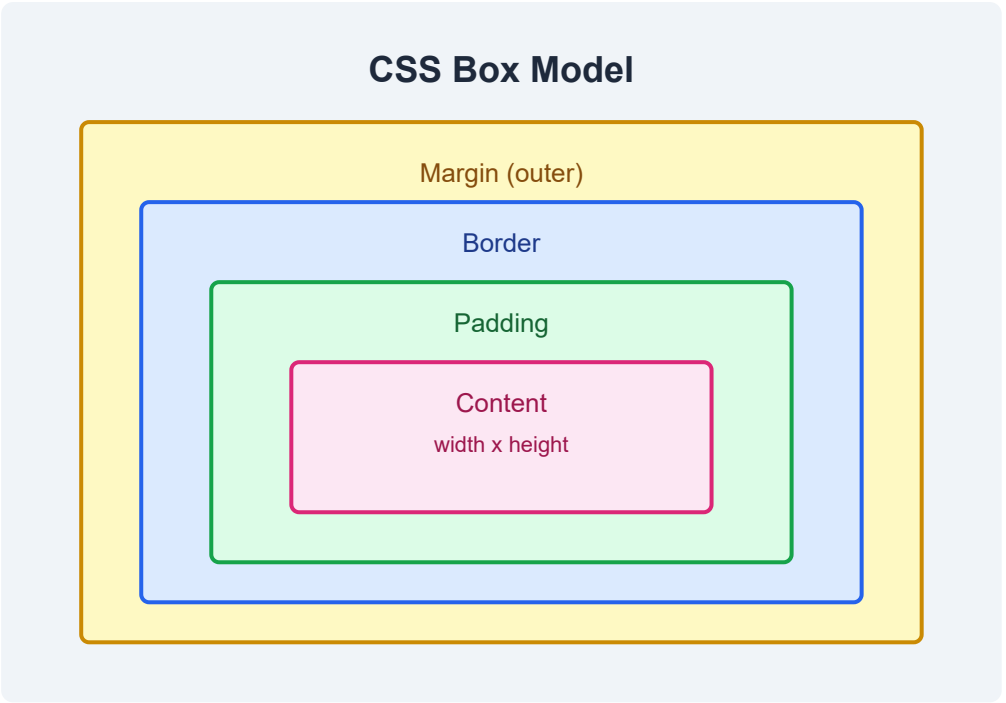
- `px` — absolute pixels
- `em` — relative to parent font-size
- `rem` — relative to root font-size
- `%` — relative to parent element
- `vw` — 1% of viewport width
- `vh` — 1% of viewport height

Specificity and !important

Specificity determines which rule wins when multiple rules target the same element. Inline styles beat IDs, IDs beat classes, classes beat elements.

```
p { color: black; }          /* specificity: 1 */
.text { color: blue; }       /* specificity: 10 */
#intro { color: green; }     /* specificity: 100 */
<p style="color: red;">      /* specificity: 1000 */
```

Avoid !important: It breaks the natural cascade and makes debugging difficult. Use specificity instead.



Practice Task

Build a horizontal navigation bar with a dropdown submenu. Style a contact form with labeled inputs, a textarea, and a styled submit button.

CHAPTER 7

Optimization & Accessibility

CSS Math Functions

calc()

Performs calculations with mixed units.

```
div {
  width: calc(100% - 40px);
  height: calc(100vh - 80px);
}
```

min() and max()

Pick the smallest or largest value from a list.

```
div {
  width: min(800px, 100%);
  font-size: max(1rem, 2.5vw);
}
```

clamp()

Clamps a value between a minimum and maximum.

```
p {
  font-size: clamp(1rem, 2.5vw, 2rem);
}
```

> **Tip:** `clamp()` is perfect for fluid typography that scales gracefully across screen sizes.

Website Optimization

Minify CSS

Remove whitespace, comments, and unnecessary characters to reduce file size.

```
/* Before */
h1 { color: blue; font-size: 24px; }

/* After minified */
h1{color:blue;font-size:24px;}
```

Reduce HTTP Requests

- Combine multiple CSS files into one.
- Use CSS sprites for small icons.
- Inline critical CSS in the `<head>` for above-the-fold content.

Accessibility

Contrast

Ensure sufficient contrast between text and background colors (WCAG AA requires a ratio of at least 4.5:1).

```
body {
  color: #1a1a1a;
  background: #ffffff;
}
```

Focus Styles

Always provide visible focus indicators for keyboard users.

```
a:focus,
button:focus {
  outline: 2px solid #3b82f6;
  outline-offset: 2px;
}
```

Screen Reader Text

Hide text visually while keeping it accessible to screen readers.

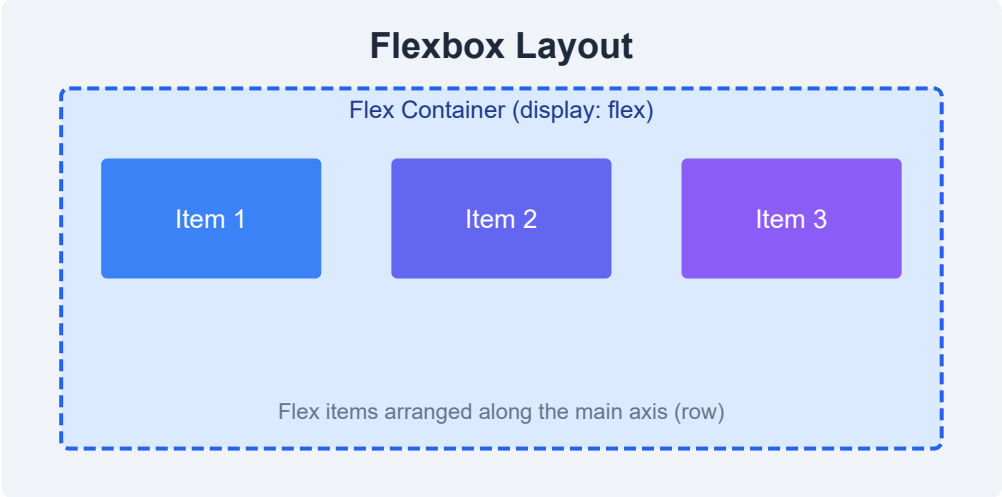
```
.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
}
```

Semantic Website Layout

Use semantic HTML elements and style them with CSS for a clear, accessible page structure.

```
<header>
  <nav>...</nav>
</header>
<main>
  <article>...</article>
  <aside>...</aside>
</main>
<footer>...</footer>
```

```css
body {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}

main {
  display: grid;
  grid-template-columns: 1fr 300px;
  gap: 20px;
  max-width: 1200px;
  margin: 0 auto;
  padding: 20px;
}

footer {
  margin-top: auto;
}
```



Practice Task

Build a semantic page layout with `<header>`, `<nav>`, `<main>`, `<aside>`, and `<footer>`. Add skip navigation, visible focus styles, and use `clamp()` for responsive font sizing.

CHAPTER 8

Advanced Styling

So far you've learned the foundations of CSS. Now it's time to unlock the creative tools that make modern websites visually stunning: rounded corners, gradients, shadows, custom fonts, transforms, transitions, and animations.

Border Radius — Rounded Corners

The `border-radius` property rounds the corners of an element. You can set one value for all corners or individual values.

```
/* All corners */
.box { border-radius: 10px; }

/* Elliptical corners */
.box { border-radius: 50%; }

/* Individual corners: top-left top-right bottom-right bottom-left */
.box {
  border-radius: 10px 20px 30px 40px;
}
```

> **Tip:** A `border-radius: 50%` on a square element creates a perfect circle.

Border Image

The `border-image` property lets you use an image as a border around an element.

```css
.card {
  border: 10px solid transparent;
  border-image: url('border.png') 30 round;
}
```

CSS Gradients

Gradients let you display smooth transitions between two or more specified colors.

Linear Gradient

```css
.gradient-linear {
  background: linear-gradient(to right, red, yellow);
}

/* Diagonal */
.gradient-diagonal {
  background: linear-gradient(45deg, #3b82f6, #8b5cf6);
}
```

Radial Gradient

```css
.gradient-radial {
  background: radial-gradient(circle, #fff, #3b82f6);
}
```

Conic Gradient

```css
.gradient-conic {
  background: conic-gradient(red, yellow, green, blue, red);
  border-radius: 50%;
}
```

Box Shadow

Add depth to elements with `box-shadow`. The syntax is: `offset-x offset-y blur-radius spread-radius color`.

```
.shadow-light {
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}

.shadow-heavy {
  box-shadow: 0 10px 30px rgba(0,0,0,0.3);
}

/* Multiple shadows */
.shadow-multi {
  box-shadow: 0 2px 4px rgba(0,0,0,0.1),
              0 8px 16px rgba(0,0,0,0.1);
}
```

Text Shadow

```
.text-shadow {
  text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
}

.text-glow {
  text-shadow: 0 0 10px #3b82f6, 0 0 20px #3b82f6;
}
```

Text Effects

Text Overflow

```
.truncate {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

Word Wrap

```
.break-long {
  word-wrap: break-word;
  overflow-wrap: break-word;
}
```

Custom Fonts with @font-face

Use `@font-face` to load custom fonts so your site isn't limited to web-safe fonts.

```
@font-face {
  font-family: 'MyCustomFont';
  src: url('fonts/myfont.woff2') format('woff2'),
       url('fonts/myfont.woff') format('woff');
  font-weight: normal;
  font-style: normal;
}

body {
  font-family: 'MyCustomFont', sans-serif;
}
```

> **Note:** Always provide fallback fonts in your `font-family` stack. Use WOFF2 format for modern browsers with WOFF as fallback.

CSS Transforms

Transform allows you to rotate, scale, translate, or skew an element.

```
.rotate { transform: rotate(45deg); }
.scale { transform: scale(1.5); }
.translate { transform: translate(50px, 100px); }
.skew { transform: skew(10deg, 5deg); }
.multiple { transform: rotate(30deg) scale(1.2) translateX(20px); }
```

CSS Transitions

Transitions smoothly change property values over a given duration.

```
.button {
  background: #3b82f6;
  transition: background 0.3s ease, transform 0.2s ease;
}

.button:hover {
  background: #2563eb;
  transform: scale(1.05);
}
```

CSS Animations with @keyframes

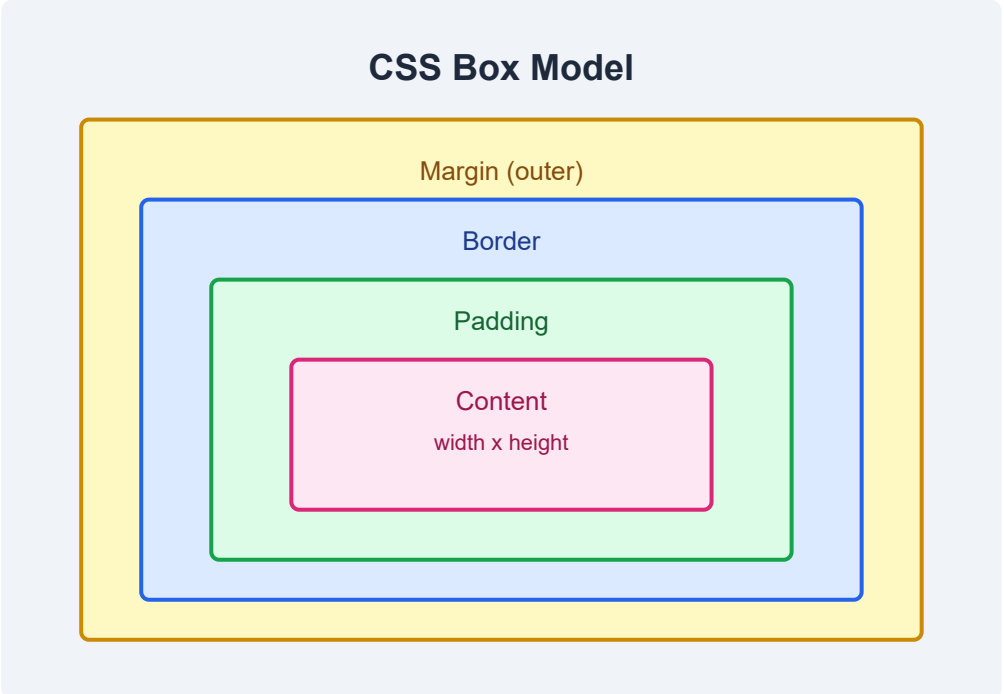
Animations let you create complex, multi-step motion sequences.

```
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(20px); }
  to   { opacity: 1; transform: translateY(0); }
}

.animated {
  animation: fadeIn 0.8s ease forwards;
}

@keyframes bounce {
  0%, 100% { transform: translateY(0); }
  50%      { transform: translateY(-20px); }
}

.bouncing {
  animation: bounce 1s ease infinite;
}
```

CSS Tooltips

Pure CSS tooltips use `::after` pseudo-elements and `:hover` to show extra information.

```css
.tooltip {
  position: relative;
  cursor: pointer;
}

.tooltip::after {
  content: attr(data-tip);
  position: absolute;
  bottom: 120%;
  left: 50%;
  transform: translateX(-50%);
  background: #1e293b;
  color: #fff;
  padding: 6px 12px;
  border-radius: 4px;
  font-size: 0.85rem;
  white-space: nowrap;
  opacity: 0;
  pointer-events: none;
  transition: opacity 0.2s;
}

.tooltip:hover::after {
  opacity: 1;
}
```



Practice Task: Create a card component that uses `border-radius`, `box-shadow`, a linear gradient background, and a `transition` that scales the card up slightly on hover. Add a tooltip to a button inside the card.

CHAPTER 9

Flexbox

Flexbox is a one-dimensional layout model that distributes space and aligns content within a container. It makes it easy to design flexible, responsive layouts without floats or positioning tricks.

The Flex Container

To create a flex layout, set `display: flex` on the parent element. The direct children become flex items.

```
.container {
  display: flex;
}
```

Flex Direction

Controls the direction flex items are placed in the container.

```
.row { flex-direction: row; }           /* default */
.column { flex-direction: column; }
.row-reverse { flex-direction: row-reverse; }
.column-reverse { flex-direction: column-reverse; }
```

Flex Wrap

By default flex items try to fit on one line. Use `flex-wrap` to allow wrapping.

```
.wrap { flex-wrap: wrap; }
.nowrap { flex-wrap: nowrap; }    /* default */
.wrap-reverse { flex-wrap: wrap-reverse; }

/* Shorthand */
.container { flex-flow: row wrap; }
```

Justify Content — Main Axis Alignment

```
.start    { justify-content: flex-start; }    /* default */
.end      { justify-content: flex-end; }
.center   { justify-content: center; }
.between  { justify-content: space-between; }
.around   { justify-content: space-around; }
.evenly   { justify-content: space-evenly; }
```

Align Items — Cross Axis Alignment

```
.stretch  { align-items: stretch; }    /* default */
.start    { align-items: flex-start; }
.end      { align-items: flex-end; }
.center   { align-items: center; }
.baseline { align-items: baseline; }
```

Align Content — Multi-line Alignment

Works when there are multiple rows of flex items (requires `flex-wrap: wrap`).

```
.container {
  display: flex;
  flex-wrap: wrap;
  align-content: space-between;
}
```

The Gap Property

Creates spacing between flex items.

```
.container {
  display: flex;
  gap: 16px;
  row-gap: 12px;
  column-gap: 24px;
}
```

Flex Item Properties

Flex Grow

Controls how much an item should grow relative to siblings.

```
.item { flex-grow: 1; }    /* all items grow equally */
.sidebar { flex-grow: 0; } /* stays at its base size */
.main { flex-grow: 2; }    /* grows twice as much */
```

Flex Shrink

Controls how much an item should shrink when there isn't enough space.

```
.item { flex-shrink: 1; }  /* default */
.no-shrink { flex-shrink: 0; }
```

Flex Basis

Sets the initial main size of a flex item before growing or shrinking.

```
.item { flex-basis: 200px; }
```

The Flex Shorthand

```
.item {
  flex: 1;              /* flex-grow: 1, flex-shrink: 1, flex-basis: 0 */
}

.item {
  flex: 0 0 auto;       /* default */
}

.item {
  flex: 1 0 300px;      /* grow, don't shrink, start at 300px */
}
```

Align Self

Overrides the container's `align-items` for a single item.

```
.item {
  align-self: flex-end;    /* or auto | flex-start | center | baseline | stre
}
```

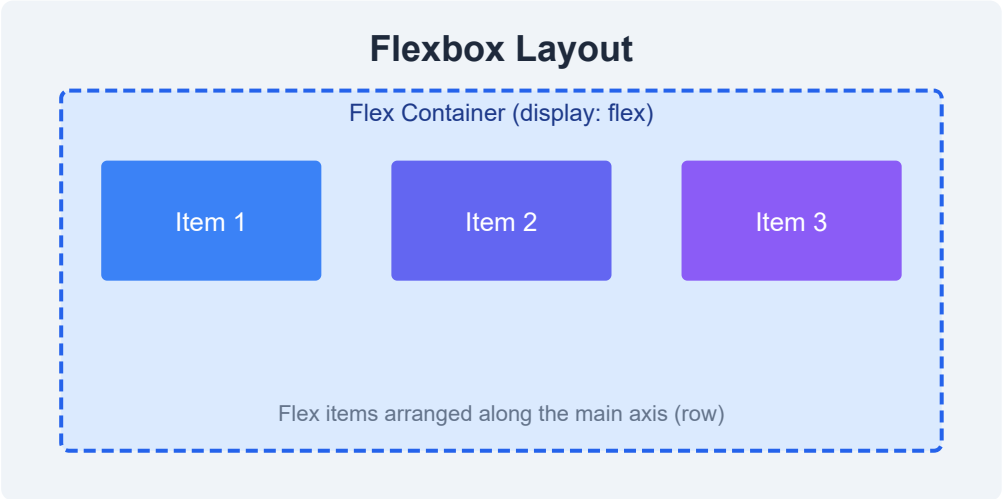
Order

Controls the visual order of flex items (default is 0).

```
.first { order: -1; }
.last  { order: 1; }
```

Responsive Flexbox Patterns

```
.card-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}

.card-grid > * {
  flex: 1 0 250px; /* grow, don't shrink, min 250px */
}

/* Holy Grail layout */
.layout {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}

.layout main {
  display: flex;
  flex: 1;
}

.layout main article { flex: 1; }
.layout main nav     { flex: 0 0 200px; }
.layout main aside   { flex: 0 0 200px; }
```



Remember: Flexbox is one-dimensional — great for rows or columns. For two-dimensional layouts, turn to CSS Grid in the next chapter.

Practice Task: Build a horizontal navigation bar using flexbox. The logo should be on the left, nav links centered, and a "Sign Up" button pushed to the right. Use `justify-content: space-between` and `align-items: center`.

CHAPTER 10

CSS Grid

CSS Grid is a two-dimensional layout system that lets you control both columns and rows at the same time. While Flexbox excels at one-dimensional layouts, Grid is the tool for building full page layouts and complex grid systems.

Setting Up a Grid Container

```
.grid-container {
  display: grid;
}
```

Grid Template Columns & Rows

Define the size and number of columns and rows.

```
.grid {
  display: grid;
  grid-template-columns: 200px 200px 200px;
  grid-template-rows: auto 200px;
}

/* Fractional units — flexible sizing */
.grid {
  grid-template-columns: 1fr 2fr 1fr;
}

/* Repeat function */
.grid {
  grid-template-columns: repeat(3, 1fr);
}

/* Mix fixed and flexible */
.grid {
  grid-template-columns: 250px 1fr 1fr;
}
```

The Gap Property

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
  row-gap: 20px;
  column-gap: 12px;
}
```

Grid Item Placement

Grid Column & Row

```
.header {
  grid-column: 1 / -1;  /* span all columns */
}

.sidebar {
  grid-row: 2 / 4;      /* span rows 2 through 4 */
}

.main {
  grid-column: 2 / 4;   /* span columns 2 and 3 */
  grid-row: 2 / 3;
}
```

Grid Area

```
.layout {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  gap: 8px;
}

.header  { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; }
.footer  { grid-area: footer; }
```

Alignment in Grid

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 100px);
  grid-template-rows: repeat(2, 100px);

  /* Align items inside their cells */
  justify-items: center;
  align-items: center;

  /* Align the entire grid inside the container */
  justify-content: center;
  align-content: center;
}

/* Per-item alignment */
.item {
  justify-self: end;
  align-self: stretch;
}
```

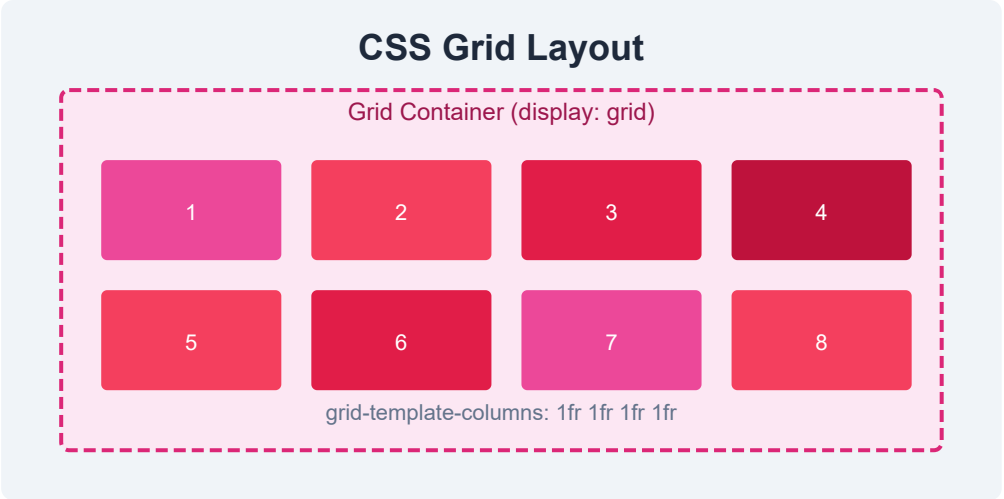
Building a 12-Column Layout System

```
.row {
  display: grid;
  grid-template-columns: repeat(12, 1fr);
  gap: 16px;
}

.col-3  { grid-column: span 3; }
.col-4  { grid-column: span 4; }
.col-6  { grid-column: span 6; }
.col-8  { grid-column: span 8; }
.col-12 { grid-column: span 12; }

/* Responsive columns */
@media (max-width: 768px) {
  .col-3, .col-4, .col-6, .col-8 {
    grid-column: span 12;
  }
}
```

Auto-fit vs Auto-fill

```
/* auto-fit — collapses empty tracks */
.grid-fit {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
}

/* auto-fill — preserves empty tracks */
.grid-fill {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
}
```

> **Key Difference:** `auto-fit` collapses empty tracks to zero width, letting items stretch. `auto-fill` keeps empty tracks at their defined size, preserving the grid structure even with fewer items.



Practice Task: Create a responsive blog layout using CSS Grid with a header, footer, main content area (spanning 8 columns), and a sidebar (spanning 4 columns). On mobile screens (below 768px), stack everything into a single column.

CHAPTER 11

Responsive Design

Responsive design ensures your website looks and works well on every device — from desktop monitors to tablets to mobile phones. It's no longer optional; it's the standard way of building for the web.

The Viewport Meta Tag

Without this tag, mobile browsers will try to render the page at a desktop width and zoom out, breaking your responsive layout.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Responsive Grid View

Build a fluid grid that adapts to the screen width.

```
.row {
  display: flex;
  flex-wrap: wrap;
}

[class*="col-"] {
  width: 100%;
  padding: 8px;
}

@media (min-width: 768px) {
  .col-1 { width: 8.33%; }
  .col-2 { width: 16.66%; }
  .col-3 { width: 25%; }
  .col-4 { width: 33.33%; }
  .col-6 { width: 50%; }
  .col-8 { width: 66.66%; }
  .col-12 { width: 100%; }
}
```

Media Queries

Media queries apply CSS only when certain conditions are true (like screen width, orientation, or resolution).

```
/* Mobile-first approach: base styles are for mobile */
.container { padding: 8px; }

/* Tablet */
@media (min-width: 768px) {
  .container { padding: 16px; }
}

/* Desktop */
@media (min-width: 1024px) {
  .container { max-width: 960px; margin: 0 auto; }
}

/* Large screens */
@media (min-width: 1440px) {
  .container { max-width: 1200px; }
}

/* Common breakpoints */
@media (max-width: 600px)  { /* phones */ }
@media (min-width: 601px) and (max-width: 1024px) { /* tablets */ }
@media (min-width: 1025px) { /* desktops */ }

/* Orientation */
@media (orientation: landscape) { /* landscape styles */ }
@media (orientation: portrait)  { /* portrait styles */ }

/* Dark mode preference */
@media (prefers-color-scheme: dark) {
  body { background: #0f172a; color: #e2e8f0; }
}
```

Responsive Images

Max-Width Approach

```
img {
  max-width: 100%;
  height: auto;
}
```

Picture Element

```
<picture>
  <source media="(min-width: 1024px)" srcset="large.jpg">
  <source media="(min-width: 768px)" srcset="medium.jpg">
  <img src="small.jpg" alt="Responsive image">
</picture>
```

Responsive Videos

```
.video-container {
  position: relative;
  padding-bottom: 56.25%; /* 16:9 aspect ratio */
  height: 0;
  overflow: hidden;
}

.video-container iframe {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
}
```

CSS Frameworks

Bootstrap

Bootstrap provides a 12-column grid system, utility classes, and pre-built components. You can use it via CDN without installing anything.

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3/dist/css/bootstrap.min

<div class="container">
  <div class="row">
    <div class="col-md-8">Main content</div>
    <div class="col-md-4">Sidebar</div>
  </div>
</div>
```

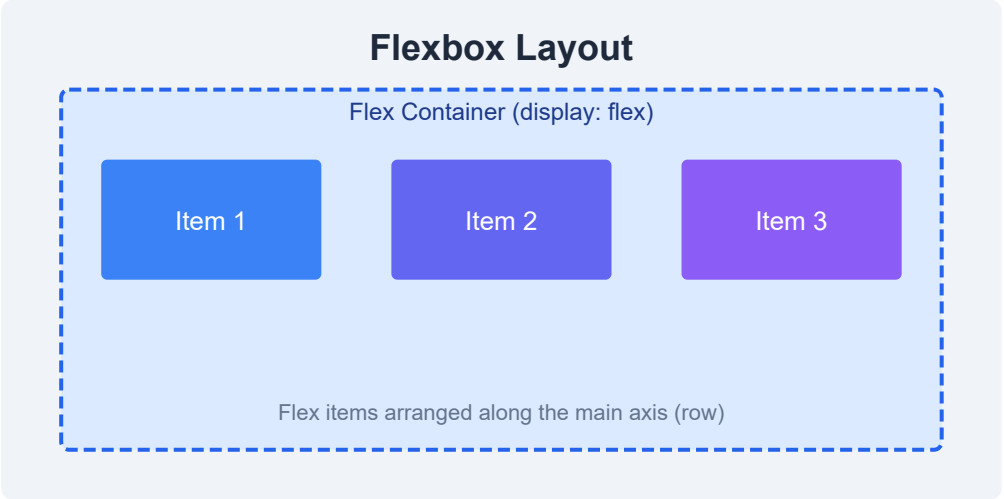
Tailwind CSS

Tailwind is a utility-first framework that lets you compose designs directly in your HTML using small utility classes.

```
<div class="flex justify-between items-center p-4 bg-blue-500 text-white roun
  <h1 class="text-2xl font-bold">Hello</h1>
  <button class="bg-white text-blue-500 px-4 py-2 rounded">Click</button>
</div>
```

Responsive Design Templates

A common responsive pattern uses a flexible container, fluid images, and media queries at breakpoints of 600px, 768px, 1024px, and 1440px. Always design mobile-first — start with the narrowest layout and add complexity as the viewport grows.



Best Practice: Don't target specific devices with your breakpoints. Instead, adjust the layout when it *looks like it needs it*. Let the content dictate the breakpoints.

Practice Task: Take a 3-column desktop layout and make it responsive. On tablets, collapse to 2 columns. On phones, stack all columns vertically. Use a mobile-first approach with `min-width` media queries.

CHAPTER 12

CSS Preprocessors — SASS

CSS preprocessors extend CSS with variables, nesting, mixins, functions, and more. They compile into standard CSS that browsers can understand. SASS (Syntactically Awesome Style Sheets) is the most popular preprocessor.

What is SASS?

SASS is a CSS preprocessor that adds programming-like features to CSS. It helps you write cleaner, more maintainable, and reusable stylesheets. SASS files end in `.scss` (modern syntax) or `.sass` (indented syntax).

SCSS vs SASS Syntax

SCSS (Sassy CSS)

Uses braces and semicolons — just like regular CSS. Any valid CSS is valid SCSS.

```
.card {
  background: #fff;
  border-radius: 8px;
  padding: 16px;
}
```

SASS (Indented Syntax)

Uses indentation instead of braces and semicolons.

```
.card
  background: #fff
  border-radius: 8px
  padding: 16px
```

> **Recommendation:** Start with `.scss`. It's closer to regular CSS and has better tooling support. You can always try the indented syntax later.

Variables

Store values like colors, fonts, and sizes in variables and reuse them throughout your stylesheet.

```
$primary: #3b82f6;
$secondary: #8b5cf6;
$font-stack: 'Segoe UI', sans-serif;
$padding-base: 16px;

.button {
  background: $primary;
  font-family: $font-stack;
  padding: $padding-base $padding-base * 2;
}

.button--secondary {
  background: $secondary;
}
```

Nesting Rules

Nest selectors inside one another to mirror the HTML structure.

```
.card {
  background: #fff;
  border-radius: 8px;
  padding: 16px;

  .card__title {
    font-size: 1.25rem;
    font-weight: 700;
  }

  .card__body {
    color: #64748b;
    line-height: 1.6;
  }

  /* Use & to reference the parent */
  &:hover {
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
  }
}
```

Partials, @import, and @use

Split your styles into partial files (starting with `_`) and import them.

```
// _variables.scss
$primary: #3b82f6;
$secondary: #8b5cf6;

// _buttons.scss
.button { background: $primary; }

// main.scss (@use is the modern replacement for @import)
@use 'variables';
@use 'buttons';

// @import still works but is deprecated
// @import 'variables';
```

Mixins (@mixin / @include)

Mixins let you define reusable blocks of styles. They can accept arguments.

```
@mixin flex-center {
  display: flex;
  align-items: center;
  justify-content: center;
}

@mixin respond-to($breakpoint) {
  @if $breakpoint == tablet {
    @media (min-width: 768px) { @content; }
  } @else if $breakpoint == desktop {
    @media (min-width: 1024px) { @content; }
  }
}

.navbar {
  @include flex-center;
}

.sidebar {
  @include respond-to(tablet) {
    width: 250px;
  }
}
```

SASS Functions

```
$base-size: 16px;

@function rem($px) {
  @return $px / $base-size * 1rem;
}

.heading {
  font-size: rem(32);  /* outputs 2rem */
}

// Built-in color functions
.notification {
  background: $primary;
  border: 1px solid darken($primary, 10%);
  color: lighten($primary, 60%);
}
```

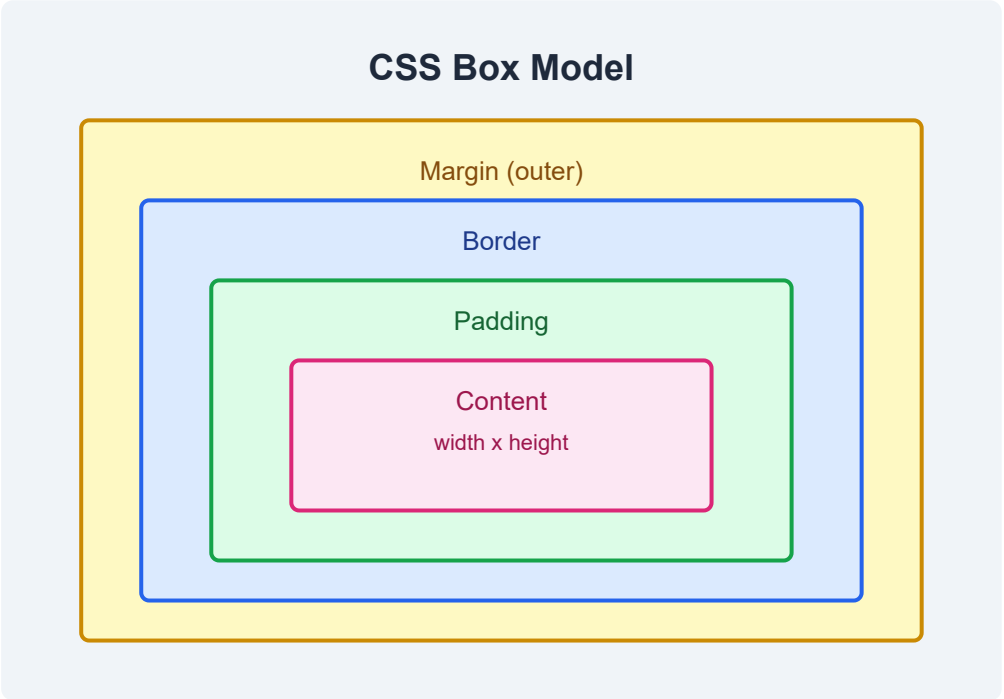
Inheritance with @extend

Share styles between selectors to avoid duplication.

```
%message-shared {
  padding: 12px;
  border-radius: 4px;
  font-weight: 500;
}

.success { @extend %message-shared; background: #22c55e; }
.error   { @extend %message-shared; background: #ef4444; }
.warning { @extend %message-shared; background: #eab308; }
```

Number & Color Operations

```
// Arithmetic
$width: 200px;
.half { width: $width / 2; }
.double { width: $width * 2; }

// Color operations
$brand: #3b82f6;
.darker  { color: $brand - #222; }
.lighter { color: $brand + #444; }
```



Tooling: To compile SASS to CSS, install the SASS package: `npm install -g sass`, then run `sass input.scss output.css` or use a build tool like Vite or Webpack that handles it automatically.

Practice Task: Convert a plain CSS file into SASS. Create a `_variables.scss` partial for colors and fonts, a `_mixins.scss` partial with a responsive mixin, and a `main.scss` that uses both. Compile it to CSS and verify the output.

CHAPTER 13

Practice & Certification

You've learned the fundamentals of CSS. Now it's time to put it all together with real projects, challenges, and a clear path toward certification and career readiness.

CSS Examples & Templates

Study these common patterns to understand how professional CSS is written:

```
/* Card component */
.card {
  background: #fff;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
  overflow: hidden;
}

.card__image { width: 100%; height: 200px; object-fit: cover; }
.card__body   { padding: 16px; }
.card__title  { font-size: 1.25rem; margin-bottom: 8px; }
.card__text   { color: #64748b; line-height: 1.6; }

/* Button system */
.btn {
  display: inline-block;
  padding: 10px 20px;
  border: none;
  border-radius: 6px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.2s;
}

.btn-primary { background: #3b82f6; color: #fff; }
.btn-primary:hover { background: #2563eb; }
.btn-secondary { background: #e2e8f0; color: #1e293b; }

/* Centered layout */
.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 16px;
}
```

Online Editors

- **CodePen** — Great for quick prototypes and sharing code snippets
- **JSFiddle** — Lightweight editor with collaboration features
- **CodeSandbox** — Full IDE in the browser, supports build tools
- **StackBlitz** — Fast, online VS Code experience

Useful CSS Snippets

```
/* Smooth scrolling */
html { scroll-behavior: smooth; }

/* Aspect ratio boxes */
.aspect-16-9 { aspect-ratio: 16 / 9; }

/* Truncate text to one line */
.ellipsis {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

/* Custom scrollbar */
::-webkit-scrollbar { width: 8px; }
::-webkit-scrollbar-track { background: #f1f5f9; }
::-webkit-scrollbar-thumb { background: #94a3b8; border-radius: 4px; }

/* Smooth image loading */
img { content-visibility: auto; }
```

CSS Quiz — Test Your Knowledge

1. What does `display: flex` do to child elements?
2. What is the difference between `justify-content` and `align-items`?
3. How do you create a CSS Grid with three equal columns?
4. What does `box-sizing: border-box` do?
5. How do you make a layout responsive with media queries?
6. What is the difference between `auto-fit` and `auto-fill` in Grid?
7. How do you define a CSS variable and use it?
8. What are pseudo-classes? Name three examples.

Exercises — Increasing Difficulty

Beginner

Style a personal profile card with a photo, name, bio, and social links using Flexbox. Center it on the page.

Intermediate

Build a responsive blog homepage with a header, featured posts grid (using CSS Grid), a sidebar with categories, and a footer. Use media queries for mobile layout.

Advanced

Create a product landing page with a fixed navigation bar, hero section with gradient background, animated elements using `@keyframes`, a pricing table with 3 columns, and a contact form with styled inputs. Make the entire page responsive and use SASS (.scss) for the styles.

Building a Full Website Project

Combine everything you've learned into a multi-page website:

1. **Home page** — Hero, featured sections, testimonials
2. **About page** — Team grid, timeline
3. **Services page** — Pricing cards with hover effects
4. **Blog page** — Post grid with Flexbox/Grid
5. **Contact page** — Styled form with validation

Study Plan

Week	Focus
1	Selectors, box model, colors, backgrounds
2	Layout with Flexbox & Grid
3	Responsive design, media queries
4	Animations, transitions, transforms
5	SASS, advanced selectors
6	Build a full project

Interview Preparation

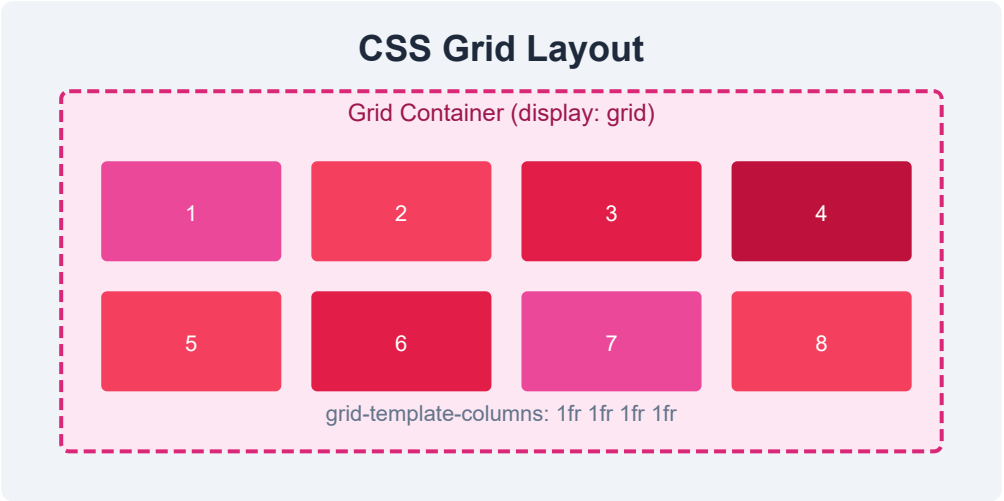
Common CSS Interview Questions

- Explain the box model and how `box-sizing` affects it.
- What's the difference between Flexbox and CSS Grid? When would you use each?
- How do you center a div both horizontally and vertically?
- Explain CSS specificity. How does the cascade work?
- What are CSS custom properties (variables)? How do they differ from SASS variables?
- How does the `position` property work? Compare `relative`, `absolute`, `fixed`, and `sticky`.
- What are the different ways to include CSS in a page?

Certification Paths

- **W3Schools CSS Certificate** — Good for beginners, validates basic CSS knowledge
- **freeCodeCamp Responsive Web Design** — Free, project-based, 300 hours

- **Microsoft Exam 70-480 (legacy)** — Programming in HTML5 with JavaScript and CSS3
- **LinkedIn Skill Assessments** — Free CSS skill assessment with verified badge
- **Frontend Masters / Pluralsight** — In-depth video courses with assessments



Final Advice: The best way to learn CSS is to build projects. Start small, copy designs you like, and gradually take on harder challenges. Read other people's code, use browser dev tools to inspect styles, and never stop experimenting.

Practice Task: Choose a real website you use daily and rebuild its homepage with HTML and CSS from scratch. Don't look at their source code — try to reproduce the layout, colors, spacing, and effects by eye. This is the single best exercise to level up your CSS skills.

CHAPTER 14

CSS References

This reference chapter compiles all the CSS selectors, properties, at-rules, functions, units, and color formats into one place. Use it as a quick lookup while building your projects.

Complete Selector Reference

Selector	Example	Description
`.class`	`.intro`	Selects all elements with class="intro"
`#id`	`#header`	Selects the element with id="header"
`*`	`*`	Universal selector — selects all elements
`element`	`p`	Selects all <p> elements
`[attr]`	`[target]`	Selects elements with a target attribute
`:pseudo`	`:hover`	Pseudo-class selector
`::pseudo`	`::before`	Pseudo-element selector

Combinators

Combinator	Symbol	Example	Description
Descendant	space	`div p`	Selects all <p> inside
Child	`>`	`div > p`	Direct child of
Adjacent sibling	`+`	`div + p`	First <p> immediately after
General sibling	`~`	`div ~ p`	All <p> siblings after

Pseudo-classes

```
:hover         /* mouse over */
:focus         /* element focused */
:active        /* being clicked */
:visited       /* visited link */
:link          /* unvisited link */
:first-child   /* first child of parent */
:last-child    /* last child of parent */
:nth-child(n)  /* nth child */
:nth-of-type(n)/* nth of its type */
:not(s)        /* not matching selector */
:checked       /* checked input */
:disabled      /* disabled input */
:empty         /* element with no children */
:root          /* root element (html) */
```

Pseudo-elements

```
::before       /* insert content before element */
::after        /* insert content after element */
::first-line   /* style first line of text */
::first-letter /* style first letter */
::selection    /* portion selected by user */
::placeholder  /* input placeholder text */
```

CSS At-rules

At-rule	Purpose
`@media`	Applies styles based on media queries (screen size, orientation, etc.)
`@keyframes`	Defines animation keyframe sequences
`@font-face`	Loads custom fonts
`@supports`	Feature detection — applies styles if browser supports a property
`@import`	Imports another stylesheet (use with caution — can slow rendering)
`@layer`	Declares a cascade layer for controlling specificity order

CSS Functions

```
url()          /* load a resource (image, font, etc.) */
rgb() / rgba() /* RGB color values */
hsl() / hsla() /* HSL color values */
calc()         /* mathematical calculation */
min()          /* smallest value from a list */
max()          /* largest value from a list */
clamp()        /* clamp a value between min and max */
minmax()       /* used in Grid for min/max track sizes */
repeat()       /* repeat grid tracks */
var()          /* reference a CSS custom property */
attr()         /* use an HTML attribute value in CSS */
```

Web Safe Fonts

```
font-family: Arial, Helvetica, sans-serif;
font-family: 'Times New Roman', Times, serif;
font-family: 'Courier New', Courier, monospace;
font-family: Georgia, 'Times New Roman', serif;
font-family: Verdana, Geneva, sans-serif;
font-family: 'Trebuchet MS', sans-serif;
font-family: Impact, Haettenschweiler, sans-serif;
font-family: 'Segoe UI', Tahoma, Geneva, sans-serif;
```

Animatable Properties

Most CSS properties that accept numeric values, colors, or transforms can be animated. Common examples:

```
opacity, color, background-color, transform
width, height, margin, padding, border
top, right, bottom, left
font-size, letter-spacing, line-height
box-shadow, text-shadow, filter
flex-grow, flex-shrink, gap
```

CSS Units Table

Unit	Type	Description
`px`	Absolute	1 pixel = 1/96th of 1 inch
`em`	Relative	Relative to parent element's font-size
`rem`	Relative	Relative to root element's font-size (html)
`%`	Relative	Percentage of parent element
`vw`	Viewport	1% of viewport width
`vh`	Viewport	1% of viewport height
`fr`	Fraction	Fraction of available space in Grid
`cm / mm / in`	Absolute	Physical units (print stylesheets)

PX to EM Converter

The formula is: `target / parent = value`. If the parent has a font-size of 16px (browser default) and you want 32px, then `32 / 16 = 2em`. For rem, it's always relative to the root `<html>` element's font-size (typically 16px).

```
/* Assuming 16px base */
8px  = 0.5rem
12px = 0.75rem
16px = 1rem
24px = 1.5rem
32px = 2rem
48px = 3rem
64px = 4rem
```

Color Formats

```
/* Named colors */
color: red;
color: crimson;

/* Hexadecimal */
color: #ff0000;        /* 6-digit */
color: #f00;           /* 3-digit shorthand */
color: #ff000080;      /* 8-digit with alpha */

/* RGB / RGBA */
color: rgb(255, 0, 0);
color: rgba(255, 0, 0, 0.5);

/* HSL / HSLA */
color: hsl(0, 100%, 50%);
color: hsla(0, 100%, 50%, 0.5);

/* Modern — OKLCH and LAB */
color: oklch(0.7, 0.2, 30);
```

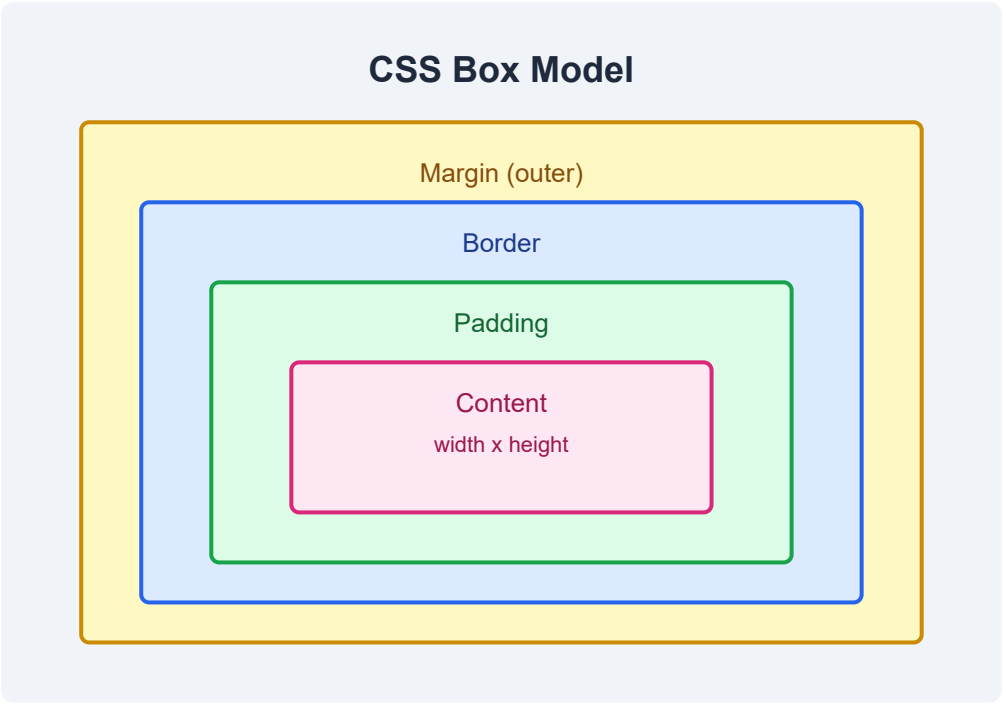
Default Browser Styles

Browsers apply default styles to HTML elements. For example, `<body>` has a default margin of 8px, headings have bold font-weight and specific font-sizes, and links are blue and underlined. CSS resets or Normalize.css are commonly used to create a consistent baseline across browsers.

```
/* Simple CSS reset */
*,
*::before,
*::after {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

Browser Support Information

Always check Can I Use before using a cutting-edge CSS feature. Modern CSS features like Grid, Flexbox, custom properties, and `clamp()` have excellent browser support. Older features like `border-image` have more limited support. Vendor prefixes (`-webkit-` , `-moz-`) are rarely needed today but may appear in legacy codebases.



> **Pro Tip:** Bookmark MDN CSS Reference — it's the most authoritative and up-to-date resource for CSS documentation.

> **Practice Task:** Create your own CSS cheat sheet as an HTML page. Include all selector types, the box model diagram, a color format converter table, and common layout patterns. Use this chapter as a starting template.